Міністерство освіти і науки України

Національний технічний університет України «Київський політехнічний інститут імені Ігоря Сікорського"

Факультет інформатики та обчислювальної техніки

Кафедра ІПІ

Звіт

з лабораторної роботи № 7 з дисципліни

«Основи програмування. Частина 2. Модульне програмування»

„ПОБУДОВА ТА ВИКОРИСТАННЯ СТРУКТУР ДАНИХ”

Виконав(ла) *ІП-56 Кліменко Ілля Сергійович*

(шифр, прізвище, ім'я, по батькові)

Перевірив *Куценко Микита Олександрович*

(прізвище, ім'я, по батькові)

Київ 2026

Мета лабораторної роботи: дослідити типи лінійних та нелінійних структур даних, навчитись користуватись бібліотечними реалізаціями структур даних та будувати власні.

Завдання:

№	Тип даних елементів	Тип списку	Спосіб додавання елементу списку	Операції зі списком
1	char	Одно спрямова ний	Включення до початку	1.Знайти перше входження символу «!». 2.Знайти суму елементів, значення яких більше за значення знайденого елементу (нумерація починається з голови списку). 3.Отримати новий список зі значень елементів, які знаходяться після символу «!», знайти їх кількість. 4.Видалити всі символи, що розташовані до символу «!» (нумерація починається з голови списку).

Виконана робота розміщена на GitHub за посиланням:

https://github.com/illya4git/LinkedListTask

Приклад роботи:

```
Run    LinkedListTask

C:/Users/Illya/RiderProjects/LinkedListTask/LinkedListTask/bin/Debug/net8.0/LinkedListTask.exe
--- Заповнення списку ---
Послідовно введіть усі символи. Натисніть Enter після порожнього вводу аби зупинитись.
Символ: H
Символ: e
Символ: l
Символ: l
Символ: o
Символ: !
Символ: o
Символ: l
Символ: l
Символ: e
Символ: H
Символ:

Початковий список: H -> e -> l -> l -> o -> ! -> o -> l -> l -> e -> H

--- Маніпуляції зі списком ---
Уведіть шуканий символ: !

1. Перше входження символу '!' знайдено на позиції 6.
2. Сума ASCII-кодів для більших елементів: 1000.
3. Кількість елементів після символу '!': 5.
   Новий список: H -> e -> l -> l -> o
4. Список після видалення елементів перед символом '!': ! -> o -> l -> l -> e -> H
```

Висновок:

Програма працює коректно та повністю відповідає вимогам завдання.

Відповіді на контрольні запитання:

1. *Що таке структура даних?*

   Це спеціалізований формат організації, управління та зберігання даних, який дозволяє ефективно отримувати до них доступ та модифікувати їх.

2. *Наведіть типи структур даних.*

   Лінійні (елементи розташовані послідовно: масиви, списки, стеки, черги), нелінійні (дані мають ієрархічні або мережеві зв'язки: дерева, графи), хешовані (побудовані на прямому доступі за ключем: хеш-таблиці).

3. *Поясніть, чим відрізняється список від словника.*

   У списку доступ здійснюється за порядковим індексом, а у словнику — за унікальним ключем.

4. *Поясніть, чим відрізняється стек від черги.*

   Стек працює за принципом «останній прийшов — першим пішов» (LIFO), а черга — «перший прийшов — першим пішов» (FIFO).

5. *Поясніть сутність побудови дерева.*

   Дерево — це ієрархічна структура з одного кореневого вузла та його нащадків.

6. **Поясніть особливості hash-таблиць.*

   Вона використовує хеш-функцію для миттєвого пошуку за ключем, долаючи колізії спеціальними методами.

7. *Поясніть сутність зв'язаної реалізації структур даних.*

   Зв'язана реалізація базується на вузлах, що зберігають дані та посилання на адреси інших вузлів у пам'яті.

8. *Наведіть приклад опису типу вузла лінійного списку у зв'язаній реалізації та поясніть устрій вузла.*

Вузол — об'єкт, що містить значення (Data) та вказівник на наступний елемент (Next).

9. *Наведіть приклад опису односпрямованого та двоспрямованого списків та поясніть різницю.*

Дозволяє рух лише вперед, тоді як двоспрямований має посилання і на наступний, і на попередній вузли.

10. *Наведіть приклади реалізації структур даних у C#.*

```
public class Node
{
    public char Data { get; set; }
    public Node Next { get; set; }

    public Node(char data)
    {
        Data = data;
        Next = null;
    }
}

public class LinkedList
{
    private Node head;

    public LinkedList()
    {
        head = null;
    }

    public void Prepend(char data)
    {
        Node newNode = new Node(data);
        newNode.Next = head;
        head = newNode;
    }
}
```